Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЕТ

к лабораторной работе №8

на тему:

«СТЕГАНОГРАФИЧЕСКИЕ МЕТОДЫ»

БГУИР 6-05-0612-02 73

Выполнил студент группы 353504
ЛЕВШУКОВ Дмитрий Александрович

Проверил ассистент каф. Информатики
ПОЛЯКОВ Кирилл Борисович

Минск 2026

СОДЕРЖАНИЕ

Введение 3
1 Выполнение работы 4
Вывод 6
Приложение А (обязательное) Исходный код программы 7

ВВЕДЕНИЕ

Стеганография – это область информационных технологий, изучающая методы скрытой передачи информации путём сокрытия самого факта её существования. В отличие от криптографии, которая преобразует сообщение в нечитаемый вид, стеганография позволяет скрыть сообщение внутри другого объекта, например изображения, аудиофайла или видеозаписи. При этом основная цель заключается в том, чтобы наличие скрытых данных не привлекало внимания посторонних лиц.

В рамках лабораторной работе рассматриваются стеганографические методы сокрытия текстовой информации в JPEG-изображениях с использованием частотной области. Для реализации выбран метод, основанный на дискретном косинусном преобразовании (DCT), которое позволяет представить изображение в виде частотных коэффициентов. Данное преобразование применяется при обработке JPEG-изображений и позволяет работать с информацией не непосредственно в пространстве пикселей, а в частотном представлении изображения.

В ходе работы изображение разбивается на блоки размером 8 × 8 пикселей. Для каждого блока выполняется преобразование яркости пикселей в частотные коэффициенты с помощью DCT. Затем для сокрытия информации изменяются два AC-коэффициента, то есть в зависимости от значения встраиваемого бита устанавливается определённое соотношение между ними. Перед встраиванием текстовое сообщение преобразуется в последовательность байтов, после чего каждый байт представляется восемью битами. Дополнительно в начало последовательности записывается длина сообщения, необходимая для правильного извлечения текста.

После изменения коэффициентов выполняется обратное дискретное косинусное преобразование (IDCT), позволяющее восстановить блок изображения в пространственной области. Обработанные блоки записываются в изображение, которое затем сохраняется в формате JPEG. Для извлечения сообщения выполняется чтение изображения, повторное преобразование его блоков в частотную область и сравнение выбранных коэффициентов. На основании полученных сравнений восстанавливаются биты, которые преобразуются в байты и исходный текст.

Задание:

1 Изучить теоретические сведения.

2 Реализовать программное средство, сокрытия (извлечения) текстового сообщения в (из) JPEG изображение(я) на основе метода сокрытия в частотной области изображения.

1 ВЫПОЛНЕНИЕ РАБОТЫ

Общий алгоритм сокрытия текстового сообщения в JPEG-изображении на основе дискретного косинусного преобразования представлен в виде блок-схемы на рисунке 1.1.

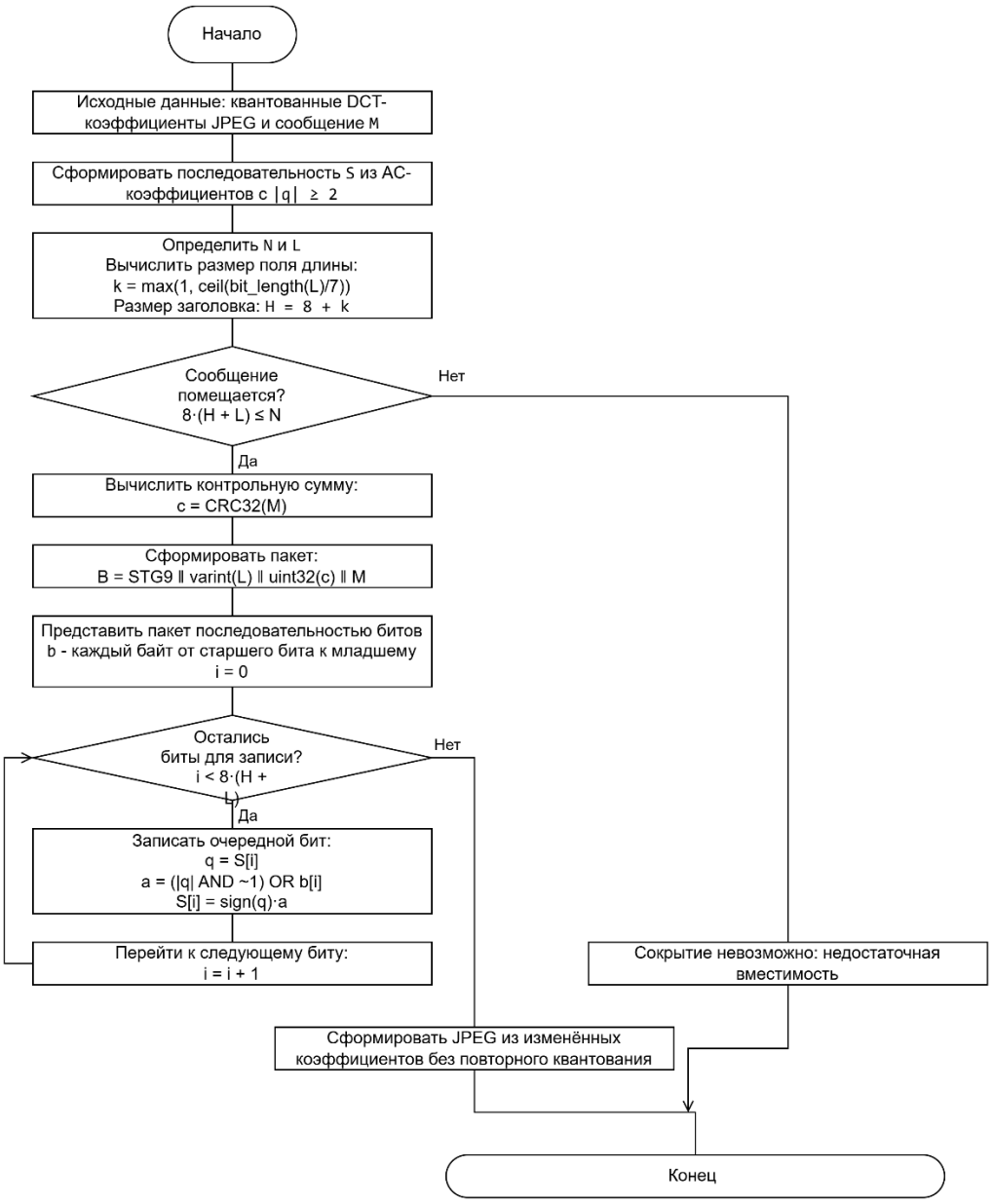


Рисунок 1.1 – Блок-схема алгоритма сокрытия на основе дискретного косинусного преобразования

Блок-схема общего алгоритма раскрытия на основе дискретного косинусного преобразования представлена на рисунке 1.2.

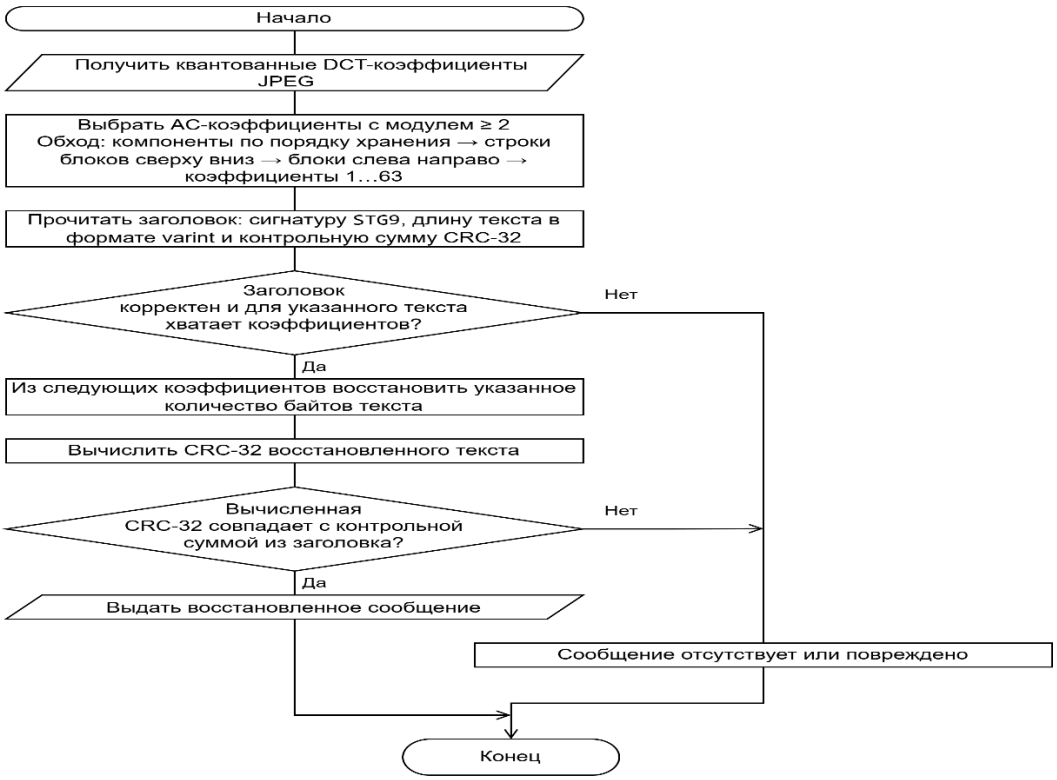


Рисунок 1.2 – Блок-схема алгоритма раскрытия на основе дискретного косинусного преобразования

Результат выполнения программы в режиме сокрытия текстового сообщения в JPEG-изображении представлен на рисунке 1.2.



Рисунок 1.2 – Результат выполнения программы в режиме сокрытия текстового сообщения

Результат выполнения программы в режиме извлечения скрытого текстового сообщения из JPEG-изображения представлен на рисунке 1.3.

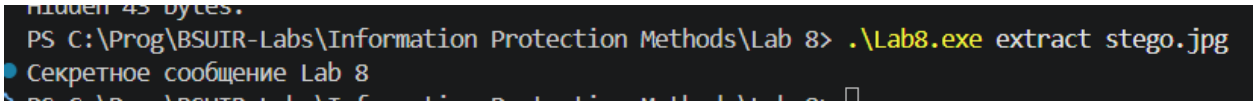


Рисунок 1.3 – Результат выполнения программы в режиме извлечения скрытого текстового сообщения

ВЫВОД

В ходе выполнения лабораторной работы были изучены принципы стеганографического сокрытия текстовой информации в частотной области JPEG-изображений, назначение дискретного косинусного преобразования и квантования коэффициентов.

На языке C++ было разработано программное средство для сокрытия и извлечения текстовых сообщений. Встраивание выполняется путём изменения чётности модулей квантованных AC-коэффициентов с сохранением их знаков. Реализованы определение вместимости изображения, компактное хранение длины сообщения в формате varint и проверка целостности текста с помощью CRC-32. Библиотека libjpeg-turbo обеспечивает чтение и запись коэффициентов без повторного квантования.

Работоспособность программы подтверждена сокрытием сообщения в JPEG-изображении и последующим извлечением с побайтовым сравнением исходного и восстановленного текста. Проверены граничные размеры сообщений и обработка некорректных данных. В результате были закреплены навыки работы со структурой JPEG и реализации стеганографического алгоритма в частотной области.

ПРИЛОЖЕНИЕ А
(обязательное)
Исходный код программы

```
#include "JpegSteganography.hpp"

#include <fstream>
#include <iostream>
#include <iomanip>
#include <iterator>
#include <stdexcept>
#include <windows.h>
#include <shellapi.h>

int main() {
    try {
        std::vector<std::string> args;
        int count = 0;
        auto wide = CommandLineToArgvW(GetCommandLineW(), &count);
        if (!wide) throw std::runtime_error("Cannot read command line.");
        for (int i = 1; i < count; ++i) {
            const int size = WideCharToMultiByte(CP_UTF8, 0, wide[i], -
1, nullptr, 0, nullptr, nullptr);
            std::string value(size, '\0');
             WideCharToMultiByte(CP_UTF8, 0, wide[i], -1, value.data(),
size, nullptr, nullptr);
            value.pop_back();
            args.push_back(value);
        }
        LocalFree(wide);
        SetConsoleOutputCP(CP_UTF8);
        if (args.size() == 1) args.insert(args.begin(), "info");
        if (args.size() < 2 ||
            !((args[0] == "info" && args.size() == 2) ||
              (args[0] == "extract" && (args.size() == 2 || args.size()
== 3)) ||
             ((args[0] == "hide" || args[0] == "hide-file") && args.size()
== 4))) {
            std::cerr << "Usage:\n"
                      << "  Lab8 info image.jpg\n"
                      << "  Lab8 image.jpg\n"
                      << "  Lab8 hide image.jpg \"message\" stego.jpg\n"
                           << "  Lab8 hide-file image.jpg message.txt
stego.jpg\n"
                      << "  Lab8 extract stego.jpg [message.txt]\n";
            return 1;
        }

        JpegSteganography image(std::filesystem::u8path(args[1]));
        if (args[0] == "info") {
            const auto bytes = image.capacity();
              std::cout << "Container: " << image.availableBits() << "
bits.\n"
                        << "Capacity: " << bytes << " message bytes (" <<
bytes * 8 << " bits)";
            if (bytes >= 1024) {
                const bool megabytes = bytes >= 1024 * 1024;
                std::cout << " = " << std::fixed << std::setprecision(2)
                          << double(bytes) / (megabytes ? 1024 * 1024 :
1024)
                          << (megabytes ? " MiB" : " KiB");
            }
```

```
                std::cout << ". Header overhead included.\n";
            } else if (args[0] == "hide" || args[0] == "hide-file") {
                std::string text = args[2];
                if (args[0] == "hide-file") {
                       std::ifstream input(std::filesystem::u8path(args[2]),
std::ios::binary);
                    if (!input) throw std::runtime_error("Cannot open message
file.");
                     text.assign(std::istreambuf_iterator<char>(input), {});
                      if (input.bad()) throw std::runtime_error("Cannot read
message file.");
                }
                image.hide(text, std::filesystem::u8path(args[3]));
                std::cout << "Hidden " << text.size() << " bytes.\n";
            } else {
                const auto text = image.extract();
                if (args.size() == 3) {
                    const auto output = std::filesystem::u8path(args[2]);
                    if (std::filesystem::exists(output) &&
                        std::filesystem::equivalent(std::filesystem::u8path(
args[1]), output))
                        throw std::runtime_error("Do not overwrite the input
image with text.");
                    std::ofstream file(output, std::ios::binary);
                     if (!file) throw std::runtime_error("Cannot open output
text file.");
                                                          file.write(text.data(),
static_cast<std::streamsize>(text.size()));
                    file.close();
                    if (!file) throw std::runtime_error("Cannot write output
text file.");
                    std::cout << "Extracted " << text.size() << " bytes.\n";
                } else {
                    std::cout << text << '\n';
                }
            }
        } catch (const std::exception& error) {
            std::cerr << "Error: " << error.what() << '\n';
            return 1;
        }
        return 0;
    }


    #pragma once

    #include <csetjmp>
    #include <cstdint>
    #include <cstdio>
    #include <filesystem>
    #include <functional>
    #include <string>
    #include <vector>
    #include <jpeglib.h>

    class JpegSteganography {
    public:
        explicit JpegSteganography(const std::filesystem::path& image);
        ~JpegSteganography();
        JpegSteganography(const JpegSteganography&) = delete;
        JpegSteganography& operator=(const JpegSteganography&) = delete;

        std::size_t capacity();
        std::size_t availableBits();
```

```
            void hide(const std::string& text, const std::filesystem::path&
output);
            std::string extract();

        private:
            struct Error {
                jpeg_error_mgr manager{};
                std::jmp_buf jump;
                char message[JMSG_LENGTH_MAX]{};
            } error_;

            static constexpr std::size_t fixedHeaderSize = 8;
            jpeg_decompress_struct reader_{};
            jpeg_compress_struct writer_{};
            jvirt_barray_ptr* blocks_ = nullptr;
            FILE* input_ = nullptr;
            FILE* output_ = nullptr;
            std::filesystem::path source_;

            static void jpegError(j_common_ptr codec);
            static std::uint32_t checksum(const std::string& text);
            static void put32(std::vector<unsigned char>& data, std::uint32_t
value);
            static std::uint32_t get32(const unsigned char* data);
            static std::size_t lengthSize(std::size_t length);
            static void putLength(std::vector<unsigned char>& data, std::size_t
length);
            static std::size_t getLength(const unsigned char* data, std::size_t
count);
            void close() noexcept;
            void visit(const std::function<bool(JCOEF&)>& action, bool writing =
false);
            void save(const std::filesystem::path& output);
        };

        #include "JpegSteganography.hpp"

        #include <cstdlib>
        #include <cstring>
        #include <limits>
        #include <stdexcept>

        namespace {
        FILE* openFile(const std::filesystem::path& path, bool writing) {
            return _wfopen(path.c_str(), writing ? L"wb" : L"rb");
        }
        }

        void JpegSteganography::jpegError(j_common_ptr codec) {
            auto* error = reinterpret_cast<Error*>(codec->err);
            codec->err->format_message(codec, error->message);
            std::longjmp(error->jump, 1);
        }

        JpegSteganography::JpegSteganography(const std::filesystem::path& image)
            : source_(image) {
            reader_.err = jpeg_std_error(&error_.manager);
            error_.manager.error_exit = jpegError;
            error_.manager.emit_message = [](j_common_ptr codec, int level) {
                if (level < 0) jpegError(codec);
            };
            if (setjmp(error_.jump)) {
                close();
               throw std::runtime_error(std::string("JPEG: ") + error_.message);
```

```
        }
        input_ = openFile(image, false);
        if (!input_) throw std::runtime_error("Cannot open input image.");
        jpeg_create_decompress(&reader_);
        jpeg_stdio_src(&reader_, input_);
        for (int marker = 0; marker < 16; ++marker)
            jpeg_save_markers(&reader_, JPEG_APP0 + marker, 0xffff);
        jpeg_save_markers(&reader_, JPEG_COM, 0xffff);
        jpeg_read_header(&reader_, TRUE);
        blocks_ = jpeg_read_coefficients(&reader_);
        if (!blocks_) {
            close();
                  throw  std::runtime_error("JPEG  does  not  contain  DCT
coefficients.");
        }
    }

    JpegSteganography::~JpegSteganography() { close(); }

    void JpegSteganography::close() noexcept {
        if (writer_.mem) jpeg_destroy_compress(&writer_);
        if (reader_.mem) jpeg_destroy_decompress(&reader_);
        if (output_) { std::fclose(output_); output_ = nullptr; }
        if (input_) { std::fclose(input_); input_ = nullptr; }
    }

    void JpegSteganography::visit(const std::function<bool(JCOEF&)>& action,
bool writing) {
        if (setjmp(error_.jump))
            throw std::runtime_error(std::string("JPEG: ") + error_.message);
        for (int c = 0; c < reader_.num_components; ++c) {
            const auto& component = reader_.comp_info[c];
            for (JDIMENSION y = 0; y < component.height_in_blocks; ++y) {
                auto row = reader_.mem->access_virt_barray(
                    reinterpret_cast<j_common_ptr>(&reader_), blocks_[c], y,
1, writing);
                for (JDIMENSION x = 0; x < component.width_in_blocks; ++x)
                    for (int k = 1; k < DCTSIZE2; ++k) {
                        auto& coefficient = row[0][x][k];
                                if  (std::abs(int(coefficient))  >=  2  &&
!action(coefficient)) return;
                    }
            }
        }
    }

    std::size_t JpegSteganography::availableBits() {
        std::size_t count = 0;
        visit([&](JCOEF&) { ++count; return true; });
        return count;
    }

    std::size_t JpegSteganography::capacity() {
        const auto bytes = availableBits() / 8;
        if (bytes < fixedHeaderSize + 1) return 0;
        auto length = bytes - fixedHeaderSize - 1;
         while (length > bytes - fixedHeaderSize - lengthSize(length)) --
length;
        return length;
    }

    std::size_t JpegSteganography::lengthSize(std::size_t length) {
        std::size_t size = 1;
        while (length >>= 7) ++size;
```

```
        return size;
    }

    void JpegSteganography::putLength(std::vector<unsigned char>& data,
std::size_t length) {
        do {
            const auto byte = static_cast<unsigned char>(length & 0x7f);
            length >>= 7;
            data.push_back(byte | (length ? 0x80 : 0));
        } while (length);
    }

    std::size_t JpegSteganography::getLength(const unsigned char* data,
std::size_t count) {
        std::size_t length = 0;
        for (std::size_t i = 0; i < count; ++i) {
            const auto value = std::size_t(data[i] & 0x7f);
            const auto shift = i * 7;
            if (shift >= std::numeric_limits<std::size_t>::digits ||
                value > (std::numeric_limits<std::size_t>::max() >> shift))
                throw std::runtime_error("Invalid message length.");
            length |= value << shift;
        }
        if (count > 1 && (data[count - 1] & 0x7f) == 0)
            throw std::runtime_error("Non-canonical message length.");
        return length;
    }

    std::uint32_t JpegSteganography::checksum(const std::string& text) {
        std::uint32_t crc = 0xffffffffu;
        for (unsigned char byte : text) {
            crc ^= byte;
            for (int bit = 0; bit < 8; ++bit)
                crc = (crc >> 1) ^ ((crc & 1) ? 0xedb88320u : 0u);
        }
        return crc ^ 0xffffffffu;
    }

    void JpegSteganography::put32(std::vector<unsigned char>& data,
std::uint32_t value) {
        for (int shift = 24; shift >= 0; shift -= 8)
            data.push_back(static_cast<unsigned char>(value >> shift));
    }

    std::uint32_t JpegSteganography::get32(const unsigned char* data) {
        std::uint32_t value = 0;
        for (int i = 0; i < 4; ++i) value = (value << 8) | data[i];
        return value;
    }

    void JpegSteganography::hide(const std::string& text, const
std::filesystem::path& output) {
        const auto bytes = availableBits() / 8;
        const auto headerSize = fixedHeaderSize + lengthSize(text.size());
        if (bytes < headerSize || text.size() > bytes - headerSize)
            throw std::runtime_error("Message is too large for this JPEG.");

        std::vector<unsigned char> data{'S', 'T', 'G', '9'};
        putLength(data, text.size());
        put32(data, checksum(text));
        data.insert(data.end(), text.begin(), text.end());

        std::size_t bit = 0;
        visit([&](JCOEF& coefficient) {
```

```
            const int value = (data[bit / 8] >> (7 - bit % 8)) & 1;
            const int magnitude = (std::abs(int(coefficient)) & ~1) | value;
            coefficient = static_cast<JCOEF>(coefficient < 0 ? -magnitude :
magnitude);
            return ++bit < data.size() * 8;
        }, true);
        save(output);
    }

    std::string JpegSteganography::extract() {
        std::vector<unsigned char> data;
                    constexpr        auto        maxLengthBytes        =
(std::numeric_limits<std::size_t>::digits + 6) / 7;
        std::size_t lengthEnd = 0;
        std::size_t bit = 0;
        visit([&](JCOEF& coefficient) {
            if (bit % 8 == 0) data.push_back(0);
            data[bit / 8] |= (std::abs(int(coefficient)) & 1) << (7 - bit %
8);
            if (++bit % 8 != 0) return true;
            if (data.size() == 4 && std::memcmp(data.data(), "STG9", 4) != 0)
                throw std::runtime_error("No Lab8 message found in this
JPEG.");
            if (data.size() > 4 && !lengthEnd) {
                if ((data.back() & 0x80) == 0) lengthEnd = data.size();
                else if (data.size() == 4 + maxLengthBytes)
                    throw std::runtime_error("Invalid message length.");
            }
            return !lengthEnd || data.size() < lengthEnd + 4;
        });
        if (!lengthEnd || bit != (lengthEnd + 4) * 8)
            throw std::runtime_error("No Lab8 message found in this JPEG.");
        const auto headerSize = lengthEnd + 4;
        const auto length = getLength(data.data() + 4, lengthEnd - 4);
        const auto expectedCrc = get32(data.data() + lengthEnd);
        const auto bytes = availableBits() / 8;
        if (headerSize > bytes || length > bytes - headerSize)
            throw std::runtime_error("Invalid message length.");

        data.assign(headerSize + std::size_t(length), 0);
        bit = 0;
        visit([&](JCOEF& coefficient) {
            data[bit / 8] |= (std::abs(int(coefficient)) & 1) << (7 - bit %
8);
            return ++bit < data.size() * 8;
        });
        std::string text(data.begin() + headerSize, data.end());
        if (checksum(text) != expectedCrc)
            throw std::runtime_error("Message checksum mismatch: data is
damaged.");
        return text;
    }

    void JpegSteganography::save(const std::filesystem::path& output) {
                        if        (std::filesystem::exists(output)        &&
std::filesystem::equivalent(source_, output))
            throw std::runtime_error("Input and output image must be different
files.");
        writer_.err = &error_.manager;
        if (setjmp(error_.jump))
            throw std::runtime_error(std::string("JPEG: ") + error_.message);
        output_ = openFile(output, true);
        if (!output_) throw std::runtime_error("Cannot open output image.");
        jpeg_create_compress(&writer_);
```

```
          jpeg_stdio_dest(&writer_, output_);
          jpeg_copy_critical_parameters(&reader_, &writer_);
          jpeg_write_coefficients(&writer_, blocks_);
          for (auto marker = reader_.marker_list; marker; marker = marker->next)
{
                const bool jfif = marker->marker == JPEG_APP0 && marker-
>data_length >= 5 &&
                                  std::memcmp(marker->data, "JFIF\0", 5) == 0;
              const bool adobe = marker->marker == JPEG_APP0 + 14 && marker-
>data_length >= 5 &&
                                   std::memcmp(marker->data, "Adobe", 5) == 0;
                  if ((jfif && writer_.write_JFIF_header) || (adobe &&
writer_.write_Adobe_marker)) continue;
             jpeg_write_marker(&writer_, marker->marker, marker->data, marker-
>data_length);
          }
          jpeg_finish_compress(&writer_);
          jpeg_destroy_compress(&writer_);
          const int result = std::fclose(output_);
          output_ = nullptr;
           if (result != 0) throw std::runtime_error("Cannot finish writing
output image.");
      }
```